

SigNoz told me "200 OK" seven times. Four spans existed.

I wanted to replay five years of SigNoz's own GitHub history into SigNoz — every pull request as a trace — so I could ask "how long do PRs really wait for review?" with a p95 instead of a vibe.

I sent seven spans. All seven came back `200 partialSuccess:{}`. Four of them existed. Nothing logged an error. No metric moved.

This is the three hours I spent finding out why, the one-line change that fixed it, and the bot that made my first dashboard a lie. If you have ever wondered whether your spans are actually arriving: go look. Don't trust the 200.

The setup

Self-hosted SigNoz v0.131.0 on AKS, collector v0.144.5. Pull the PR history from GitHub's GraphQL API, turn each PR into a trace, push it over OTLP with real historical timestamps.

The span model is the whole trick — every span's *duration* is a real wait:

```
PR #123 (first commit → merge)
├─ authoring          first commit → PR opened
├─ awaiting first review opened → first review    ← the human latency
├─ review round 1 by alice (CHANGES_REQUESTED)
└─ awaiting merge      last approval → merged
```

Because the durations are real, SigNoz's built-in aggregations do the analysis for free. I didn't have to build any of it.

Gotcha #1: the endpoint that lies

My SigNoz LoadBalancer has a public IP, so I pointed the exporter at it. `200 OK`. Zero traces.

That `200` is the React app. SigNoz's UI serves `index.html` for any unmatched path, so `/v1/traces` returns a cheerful `200 text/html` — and so does `/this/is/not/real/xyz123`:

```
$ curl -s -o /dev/null -w "%{http_code} %{content_type}\n" \
  -X POST http://<lb-ip>:8080/v1/traces
200 text/html; charset=utf-8
```

An exporter pointed there gets a success response for every batch, forever, while dropping every span. The real ingest ports (4317/4318) live on `signoz-otel-collector`, which is `ClusterIP` — not exposed at all.

Check the content-type, not the status code.

Gotcha #2: the wall at exactly 15 days

Port-forwarding to the real collector, I sent probes at different ages and checked what survived:

span age	HTTP response	in ClickHouse?
1d, 5d, 14d, 14.9d	200 partialSuccess: {}	yes
15.1d, 16d, 20d	200 partialSuccess: {}	no

A hard wall at exactly 15.0 days. The same 200 on both sides of it.

The wrong theory (that fit perfectly)

SigNoz's traces table has a TTL:

```
TTL toDateTime(timestamp) + toIntervalSecond(1296000)
```

1296000 seconds is 15 days. It's evaluated against *the span's own timestamp*, not ingest time — so a 2021 span is expired the moment it lands. The numbers matched exactly. Case closed.

It was wrong. I raised retention to six years and confirmed it in ClickHouse (`toIntervalSecond(189216000)`). The wall didn't move. Still exactly 15 days.

Two numbers were both 15 for unrelated reasons and I'd assumed causation. What killed the theory was bisecting the pipeline — writing a 730-day-old row **directly** into ClickHouse, bypassing the collector:

```
INSERT INTO signoz_traces.signoz_index_v3
(timestamp, trace_id, span_id, name, duration_nano, `resource_string_service$$name`)
VALUES (now() - INTERVAL 730 DAY, 'aa00...', '1122...', 'direct-730d', 1000000, 'direct-test');
```

It persisted. ClickHouse was happy to store a two-year-old span. The database was never the problem.

The actual answer

The collector logged nothing. Its `refused` / `dropped` metrics were zero. The pipeline had no filter processor. OpAMP reported `"Config has not changed"`.

The answer was in the source — `exporter/clickhousetracesexporter/clickhouse_exporter.go`:

```
maxAllowedDataAgeDays: 15,
```

Hardcoded. And in `clickhouse_exporter_v3.go:414`:

```
ts := uint64(span.StartTimestamp())
if ts < uint64(oldestAllowedTs) {
    s.logger.Debug("skipping span outside allowed time window", ...)
    continue
}
```

Two things make this invisible. **It logs at `Debug`**, and the collector runs at `Info`. And **it's a bare `continue`** — no metric increments, so `send_failed_spans` stays at zero because nothing "failed". The span dies in the one gap where nothing is counted: after accepted, before written.

Then I found the part that makes this more than a gotcha. The **logs** exporter has the identical constant — but exposes it:

```
defaultMaxAllowedDataAgeDays int = 15
MaxAllowedDataAgeDays *int `mapstructure:"max_allowed_data_age_days"`
```

You can backfill logs older than 15 days. You cannot backfill traces. Same constant, same purpose, one configurable and one not.

That asymmetry isn't a design decision. It's an unfinished job, and the history says so. In January 2026, someone filed [#750](https://github.com/SigNoz/signoz-otel-collector/issues/750) describing *exactly* this — for logs. Their words: "Collector metrics indicate success, but no data appears in SigNoz/ClickHouse." They found it the same way I did, by turning on debug logging and reading the source.

It was fixed the next day. [PR #746](https://github.com/SigNoz/signoz-otel-collector/pull/746) added `max_allowed_data_age_days` in **+20/−4 across six files** — every one of them under `clickhouselogsexporter/`. The traces exporter, sitting there with the same hardcoded 15, wasn't touched.

So the maintainers already agree the limit should be configurable. They merged that position six months ago. Traces just never got the same twenty lines — and as of v0.144.6, released the day before I went looking, it still hasn't.

The fix is one line

```
- maxAllowedDataAgeDays: 15,
+ maxAllowedDataAgeDays: 3650,
```

I built the patched collector and ran it on my laptop against ClickHouse over a port-forward, so I never touched the cluster. Then I ran both binaries against the same database with the same span:

collector	HTTP response	spans stored
stock (15)	200 partialSuccess:{}	0
patched (3650)	200 partialSuccess:{}	2

One constant. With Debug on, the stock binary finally admits it:

```
debug clickhousetracesexporter/clickhouse_exporter_v3.go:414
    skipping span outside allowed time window
```

Act two: it stored my data eight times

Patch working, I fired all 91,536 spans. The client was delighted: `sent 91536/91536 http=200`. The database had **445,038 rows and 52,212 distinct span IDs** — 8.1× duplication.

Writing through a `kubecttl port-forward` is far slower than the in-cluster path the default timeout assumes. Writes timed out and the exporter did what a good exporter does: it retried. Retries are at-least-once, so every timed-out batch rewrote rows that had already landed.

The fix was a 180s timeout and turning retry **off**, so a timeout becomes a visible gap instead of silent duplication.

My own `verify` command had reported "all spans accounted for" on that eight-times-too-big data, because it compared row count to expected instead of distinct span IDs. My verification tool had the same bug as the thing it was verifying: it trusted a number without checking what it meant.

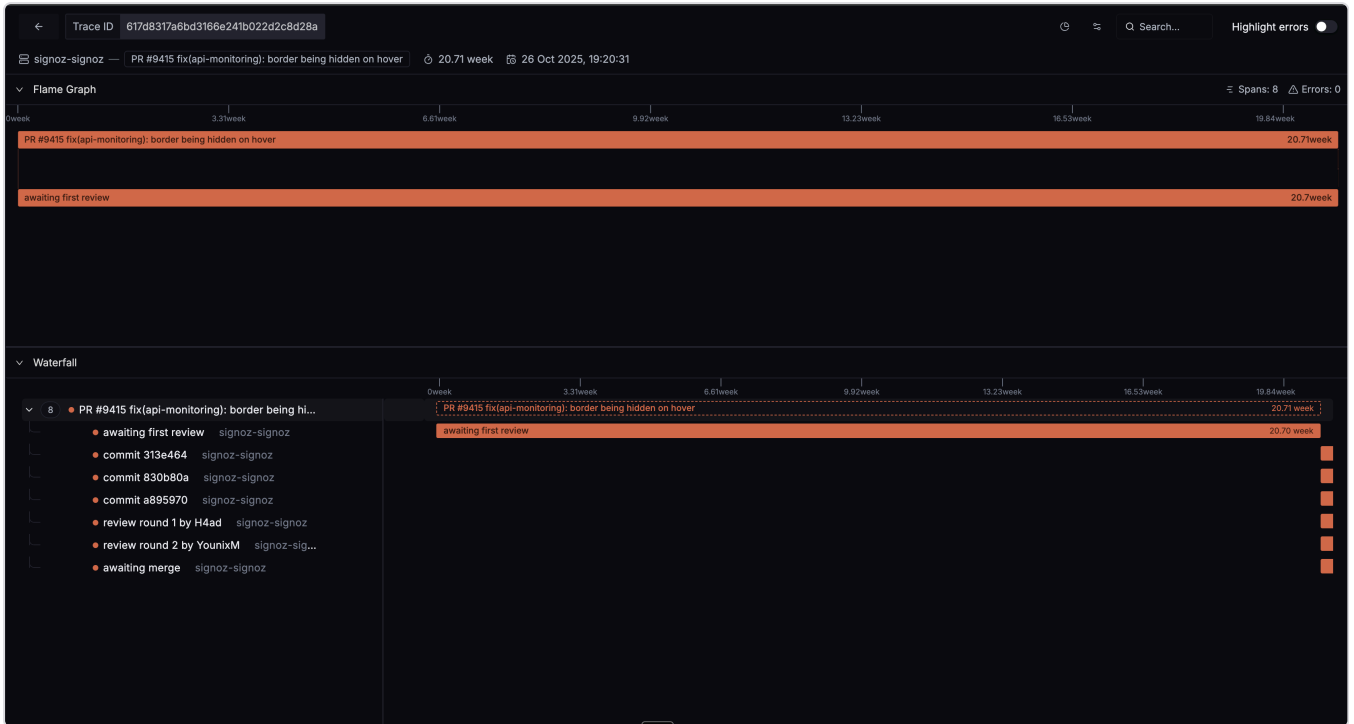
The payoff, and the bot

With the patch in, all five and a half years landed: **91,532 of 91,536 spans, 7,345 PRs, 2021-01-07 → 2026-07-15**, for about 5 MB.

Then I opened one trace:

```
PR #9415 "fix(api-monitoring): border being hidden"      145 d
├─ awaiting first review                                144.9 d
├─ review round 1 by H4ad      APPROVED                  0 m
├─ review round 2 by YounixM   APPROVED                  3.8 m
└─ awaiting merge                                           1.5 h
```

The review took four minutes. The wait took five months.

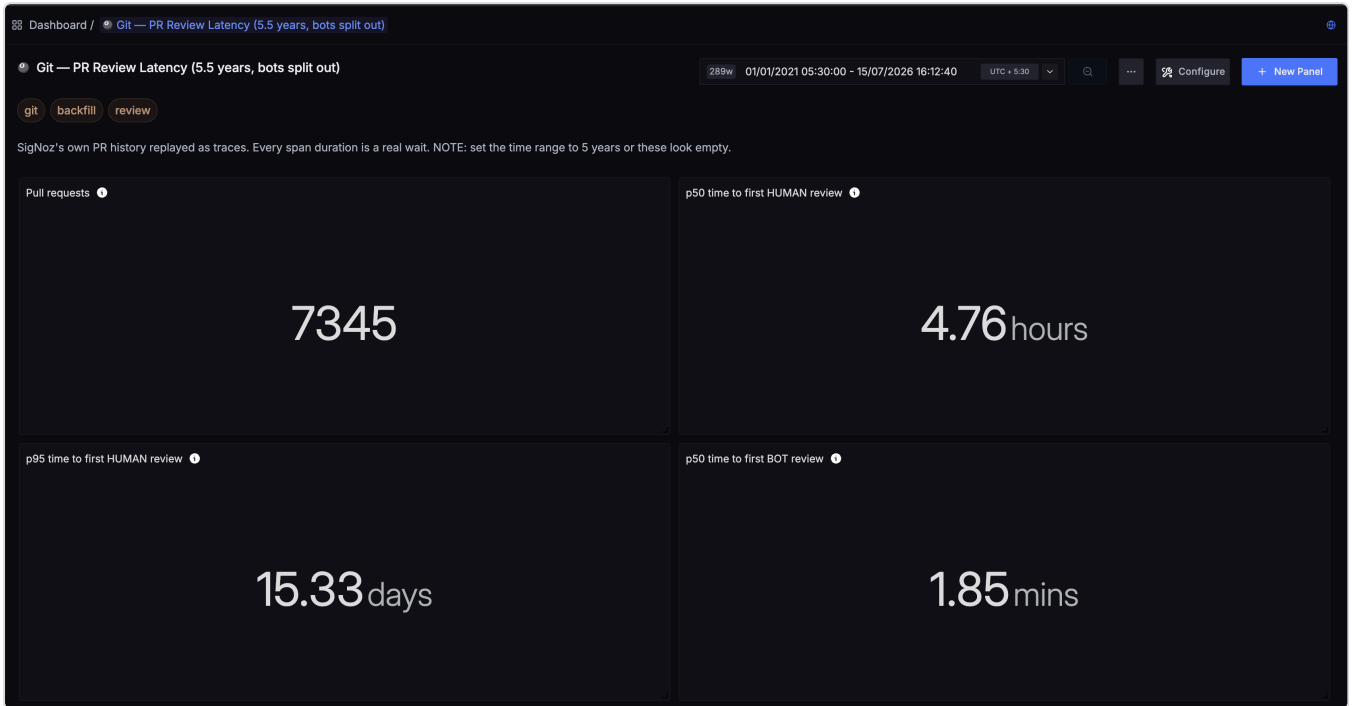


The whole trace is one bar. `awaiting first review` runs 20.70 weeks inside a 20.71-week PR; the three commits, two approvals and the merge are the slivers on the right.

My first headline was "the median PR is reviewed in 1 hour." Real number, real query, complete nonsense. `ellipsis-dev` is an AI review bot, the single largest reviewer in the repo, and it responds instantly:

reviewer	n	p50	p95
bots	1,576	1.85 min	32.8h
humans	4,667	4.76 h	15.33 days

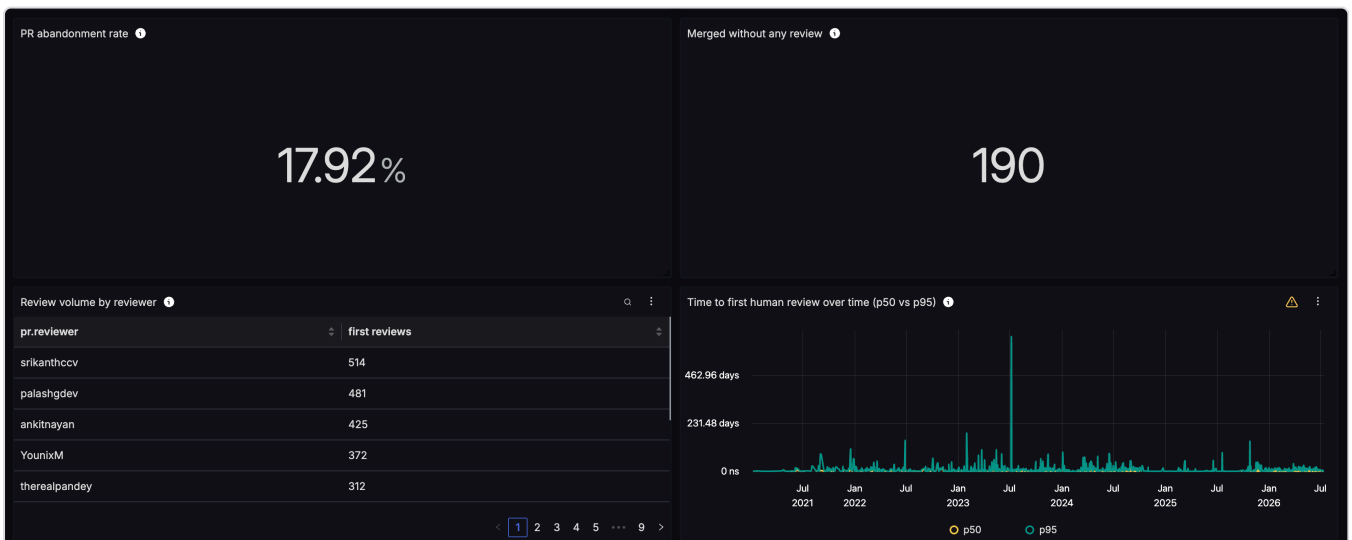
A quarter of the "reviews" weren't people. If you're a human waiting on review, 1.0h is a lie — and it's the number the obvious query hands you.



Same question, split by who answered it. The human median is 4.76 hours; the bot's is 1.85 minutes. Blend them and you get the 1.0h that means nothing.

I also nearly shipped a "slowest reviewer" leaderboard. The top entries had `n=1`, `n=2`, `n=3` — a p95 over one sample isn't a statistic, and these are real named people. Worse, the metric is backwards: `awaiting first review` grouped by reviewer measures how long the PR waited *before that person showed up*. They didn't cause the wait, they ended it. Whoever finally reviewed that 145-day-old PR rescued it. The leaderboard ranks rescuers as culprits.

Review *volume* is answerable, because it measures work actually done — so that's what the panel shows instead.



What the data can honestly say: 17.92% of PRs are abandoned, 190 merged with no review at all, and review volume by person — not speed.

The alert that can't be written

The obvious mitigation is a canary: emit a span a minute, alert when it stops. I built it. It can't work.

I made a threshold rule — count of canary spans **below 1** over 5 minutes. With no canary running, the exact condition it exists to catch, it sat `inactive`. Forever. A control rule pointed at a service that *does* have data, with a threshold guaranteed to trip, fired immediately.

The engine is fine. **"No data" isn't a low value — it's the absence of a series**, and there's nothing to compare against a threshold. If the canary were running and then died, its series would vanish and the alert would go quiet at exactly the moment it should scream.

So the drop increments no metric, and the data's absence produces no series. The failure is undetectable from inside the system by construction. The real answer is the boring one every SRE knows and I had to rediscover: a dead man's switch has to live **outside** the system it watches.

What I'd tell my past self

- `200 OK` from an OTLP endpoint means **"I parsed your protobuf."** Nothing more. The only way to know your telemetry arrived is to query it back.
- **Two equal numbers are not causation.** The TTL was 15 days and the wall was 15 days and they were unrelated. Bisecting the pipeline took ten minutes and killed a theory I'd have defended for another hour.
- **Silent drops hide at Debug.** If data vanishes with no errors, turn on debug logging before you read any more source.
- **Retry is at-least-once.** A slow sink doesn't just make things slow, it makes duplicates. Prefer a visible gap.
- **Look at who is in your data before you trust an aggregate.** A quarter of my reviewers were a bot. The p50 wasn't wrong — it was answering a different question than the one I was asking.

Conclusion

Backfilling historical traces into SigNoz is impossible out of the box, and nothing tells you: not the HTTP response, not the logs, not the metrics, not an alert. It's one hardcoded constant — one that was already made configurable for logs, six months ago, in twenty lines.

Tools are here: [repo link], including the probe that finds the wall on any OTLP endpoint. Filed upstream as [issue link], pointing at #746 as the precedent.

Verified on SigNoz v0.131.0 and signoz-otel-collector v0.144.5; the constant is unchanged on `main` and in v0.144.6. Your version may differ — run the probe and find out.